OpenIn Orchestrator

Stage-based runtime | Codex execution | Ralph story mode | Run-scoped metrics | Direct research adapters

Сервис управляет полным циклом ML/agent run: от постановки задачи и планирования до исполнения, проверки качества, упаковки результатов и повторного replanning.

Текущая архитектура ушла от monolithic runtime к явным stage handlers, persisted experiment memory и более контролируемому search loop.

CORE RUNTIME IDEA
One run, many loops

PLANNING INTELLIGENCE
Memory + Research

EXECUTION CONTRACT
Run-scoped artifacts

PROJECT SCOPE

Что делает проект

1. Управляет run lifecycle

- создает и возобновляет run
- ведет статусы и replans
- фиксирует terminal outcome

2. Планирует и исполняет шаги

- Ralph story planning
- обычный planner build/replan
- Codex и shell execution

3. Проверяет качество

- verifier и quality gate
- improvement strategy
- micro-training policy

4. Хранит историю экспериментов

- experiment attempts
- recipe snapshots
- recipe diffs

5. Подсказывает следующий ход

- structured experiment memory

CYCLES

Ключевые циклы системы

RECEIVED / CONTEXT_READY

→

PLANNING

→

EXECUTING

→

VERIFYING

→

PACKAGING

→

TERMINAL

Execution cycle

Главный replan loop: `CONTEXT_READY → PLANNING → EXECUTING → VERIFYING`. Если нужен replan, `execution_cycle` увеличивается и run возвращается в `CONTEXT_READY`.

Step loop

Внутри `EXECUTING` сервис крутит `next_step_index`, исполняет шаг, пишет `run_step`, двигается к следующему шагу

DDD

DDD-схема проекта

Presentation / Delivery

`orchestrator/app/api.py`, `streamlit_app.py`, event gateways. Здесь живут HTTP endpoints, UI и точки входа для внешних операторов и сервисов.

Application

`ProcessRunTickUseCase`, stage handlers и orchestration services. Этот слой владеет lifecycle-решениями, planning context, verification persistence flow и orchestration policy.

Domain

`domain/state_machine.py` и инварианты. Здесь нет знания о Redis, SQLite, файловой системе, Codex CLI или HTTP.

Infrastructure / Adapters

Persistence, planner adapters, Codex runner, verifier, artifact publisher, runtime composition. Этот слой связывает application с внешним миром.

Практический смысл: доменный слой остается маленьким и проверяемым, application orchestrates, infrastructure подключает транспорт, storage и tool execution.

ARCHITECTURE

Текущая архитектурная схема

Ingress

- FastAPI API
- Redis / in-memory bus
- SessionManager + runtime/service

Core orchestration

- `ProcessRunTickUseCase`
- stage handlers
- policy + guards
- Ralph scenario service

Execution + persistence

- CodexRunner / Verifier
- ArtifactPublisher
- SQLite repositories
- workspace FS

Planning intelligence

- `PlanningContextService`
- `BaselineResearchService`
- `MicroTrainingPolicyService`
- `ImprovementStrategyService`

OpenIn Orchestrator3

PLANNING INTELLIGENCE

Как сервис выбирает следующий шаг

Structured experiment memory

- latest attempt
- best attempt
- recent metric deltas
- repeated interventions
- recent hyperparameter moves
- recent recipe diffs

Baseline / research context

- direct `arXiv` lookup
- `Hugging Face Papers` trending enrichment
- optional custom backend fallback
- `pdf_fts` and heuristic fallback

Итог

Planner больше не работает как stateless editor. Он видит, что уже пробовали, какой recipe реально изменился, где был рост метрики и что подсказывает внешний research по dataset/task/metric.

EXECUTION & VERIFICATION

Контроль качества и честности обучения

Run-scoped artifacts

Verifier больше не сканирует весь workspace. Acceptance идет только по текущему `run-scoped metrics artifact`.

Policy guards

Гварды режут synthetic smoke, task reformulation, foreign metrics path, inconsistent planning-only success и прочие ложные успехи.

Quality gate

Success не засчитывается без materialized target metric, корректной intent validation и валидного acceptance artifact.

Практический эффект

Система стала устойчивее к двум классам проблем: ложным высоким метрикам из stale artifacts и «умным» подменам задачи ради удобной метрики.

PERSISTENCE

Что сохраняется и почему это важно

tasks

goal, constraints, execution mode, contract-like inputs

runs

status, plan_json, verification_json, error_message, execution_cycle

run_steps

история фактических step attempts, stdout/stderr, status

experiment_attempts

metrics, hyperparameters, strategy, `recipe_snapshot`, `recipe_diff`

Почему важен `recipe_diff`

Он превращает vague history «мы что-то уже пробовали» в явное знание «мы подняли epochs, снизили learning_rate и сменили intervention, и это дало / не дало эффект».

Почему важен `plan_json` metadata

EXTERNAL CONNECTIONS

Внешние подключения

Operational

- Streamlit UI
- HTTP / CLI clients
- Redis streams
- SQLite DB
- workspace filesystem

Execution

- Codex CLI
- shell commands inside workspace
- skills and prompt contract
- artifact publishing

Research

- `arXiv` API
- `Hugging Face Papers` API
- optional custom backend
- local PDF FTS fallback

Текущий принцип

Web research больше не завязан на обязательный отдельный backend. Базовый режим — direct adapters (`arXiv` + `HF Papers`), а custom backend остается только optional fallback.

Open in Orchestrator

UI

Как оператор видит систему

Run Monitor

- status и event feed
- plan / steps / artifacts
- task intent и structured metrics
- improvement strategy

Planning Intelligence

- `Lookup Mode`
- `Research Sources`
- `Latest Attempt`
- `Best Attempt`
- вкладки `Experiment Memory` и `Research Context`

Зачем это оператору

Видно не только что run сейчас делает, но и почему planner решил именно так: на чем основан следующий intervention и какая внешняя research-подсказка была использована.

STRENGTHS

Что уже сильное в текущей версии

1. Управляемые циклы

Lifecycle, execution replan, quality loop, micro-training ladder и Ralph loop теперь разделены концептуально и в коде.

2. Накопление памяти

Система сохраняет recipe snapshots и diffs вместо того, чтобы каждый раз думать почти с нуля.

3. Более честная верификация

Run-scoped metrics, intent validation и policy guards снижают риск ложных successes.

4. Planning intelligence в UI

Теперь reasoning planner виден оператору, а не только buried in prompts.

NEXT STEPS

Что логично делать дальше

Trajectory predictor

Отдельный advisory model, который оценивает вероятность роста/плато до следующего расширения бюджета.

Richer recipe ledger

Явный config diff и более глубокая causal память: какие именно изменения дали рост или деградацию.

Research ranking

Более умный merge `arXiv + HF Papers`, а затем optional custom backend как enterprise override.

Короткий итог

Проект уже выглядит как управляемый orchestration runtime для ML-agent loops, а не как один большой procedural script. Дальнейший рост качества теперь зависит в основном от intelligence layers, а не от базовой runtime mechanics.